

Les k plus proches voisins

20 mai 2026 – B. COLOMBEL

Sommaire

1	Étude préliminaire	2
1.1	Étude des corrélations	2
1.2	Nuages de points	2
2	Prédiction	3
2.1	Sur le graphique	3
2.2	Automatisation de la prédiction	3
2.3	Carte de prédiction	5
3	Jeu d'apprentissage et jeu de test	6

L'algorithme des k plus proches voisins appartient à la famille des algorithmes d'*apprentissage automatique* (machine learning).

L'idée d'apprentissage automatique ne date pas d'hier, puisque le terme de machine learning a été utilisé pour la première fois par l'informaticien américain Arthur Samuel en 1959. Les algorithmes d'apprentissage automatique ont connu un fort regain d'intérêt au début des années 2000 notamment grâce à la quantité de données disponibles sur internet.

L'algorithme des k plus proches voisins est un algorithme d'apprentissage *supervisé*, il est nécessaire d'avoir des données labellisées. À partir d'un ensemble E de données labellisées, il sera possible de classer (déterminer le label) d'une nouvelle donnée (donnée n'appartenant pas à E).

Objectifs.

En 1936, le biologiste américain Edgar ANDERSON a collecté les données sur 3 iris d'une même espèce. Ces trois variétés sont assez similaires et peuvent être croisées entre elles.



(a) Setosa



(b) Versicolor



(c) Virginica

FIGURE 1 – Les trois iris

Pour chaque variété étudiée, Edgar ANDERSON a mesuré (en cm) les dimensions. Les résultats sont contenus dans le fichier `iris.csv`.

Ce jeu de données est composé de 150 entrées, pour chaque entrée nous avons :

- la longueur des sépales (en cm)
- la largeur des sépales (en cm)
- la longueur des pétales (en cm)
- la largeur des pétales (en cm)
- la variété de l'iris : Iris setosa, Iris virginica ou Iris versicolor (le label du jeu de données)

Objectifs.

- Étudier ce jeu de données et « apprendre » à la machine à reconnaître des iris setosa, virginica et versicolor ;
- prédire la variété d'un nouvel iris à partir de ses dimensions en recherchant ses voisins les plus proches ;
- créer des cartes de prédiction des variétés d'iris.

Bienvenue dans le monde du *machine learning* et de l'*intelligence artificielle* !

1 Étude préliminaire

1.1 Étude des corrélations

1. Importer les bibliothèques utiles.

```
[1]: % # pour utiliser matplotlib
      % matplotlib nbagg

import numpy as np # pour les maths
import pandas as pd # pour l'analyse de données
import matplotlib.pyplot as plt # pour les graphiques
```

2. Importer le jeu de données (attention aux séparateurs et à l'encodage) Appeler `iris` la variable contenant ce jeu de données.
3. Vérifier que l'importation s'est bien déroulée.

```
[3]: iris.head()
```

```
[3]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

4. la méthode `corr()` permet de calculer la *matrice des corrélations* du jeu de données.
Quelles sont les variables les plus corrélées entre elles ? les moins corrélées ?

1.2 Nuages de points

Pour visualiser les variations par espèces, nous avons besoin de tracer les mesures. Malheureusement, les données sont quadridimensionnelles ce qui rend impossible leur tracé.

Une approche possible consiste à examiner les nuages de points pour chacune des six paires de mesures.

Par exemple, pour visualiser le nuage de points des mesures `sepal_width` en fonction de `sepal_length`, on peut exécuter le code :

```
[4]: sepal_length = iris.loc[:, "sepal_length"]
      sepal_width = iris.loc[:, "sepal_width"]
      lab = iris.loc[:, "species"]

      # Affichage des données
      for e, c in [('setosa', 'g'), ('virginica', 'r'), ('versicolor', 'b')]:
          plt.scatter(sepal_length[lab == e], sepal_width[lab == e], color = c, label = e)
```

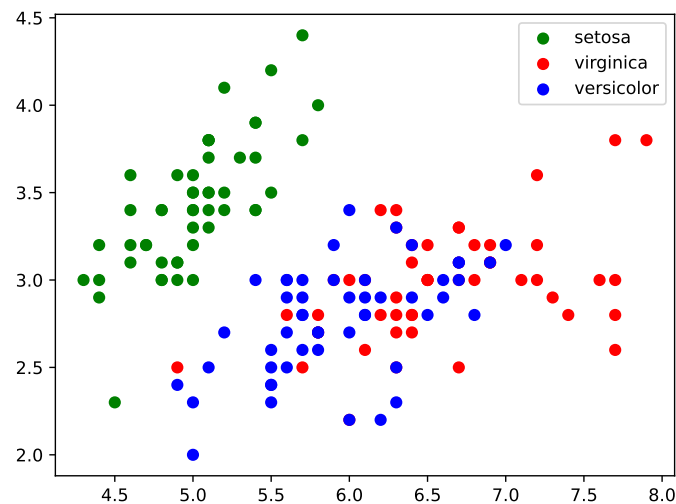


FIGURE 2 – Nuage de points des variables `sepal_length` et `sepal_width`

```
plt.legend()
plt.show()
```

On obtient le nuage de la figure 2 page 3.

1. Tracer les cinq autres nuages de points possibles.
2. D'après vous, sur quel nuage de points apparaît le plus clairement la distinction entre les trois variétés ?

2 Prédiction

Dans la suite, on travaille avec le nuage de points représentant les mesures `petal_width` en fonction de `petal_length` (voir figure 3 page 4).

2.1 Sur le graphique

1. (a) Déterminer les coordonnées du point moyen.
 (b) Transformer le graphique de la figure 3 pour que le point moyen y apparaisse (pour tracer un point de $\mathbb{R}^4$ dans le plan, il suffit de le projeter, c'est-à-dire *d'oublier* les coordonnées en trop).
 (c) Visuellement, si une iris possédait les dimensions du point moyen, quelle serait sa variété la plus probable ?
2. Reprendre les mêmes questions avec une iris dont la longueur de pétale est de 2 cm et sa largeur de 0,5 cm.
3. Peut-on être aussi catégorique avec une iris dont la longueur et la largeur de ses pétales sont respectivement 2,5 cm et 0,75 cm ?

Quand le cas ne peut être tranché, il faudrait un critère de décision qui soit

- moins subjectif que « dans un nuage » ou que « très proche »,
- algorithmique pour qu'une machine puisse décider.

Nous allons utiliser l'algorithme des k plus proches voisins.

2.2 Automatisation de la prédiction

Méthode de prédiction. Soit k un entier strictement positif.

- on calcule la distance entre le nouveau point et chaque point issu du jeu de données ;
- on sélectionne uniquement les k distances les plus petites (« les k plus proches voisins »)
- parmi les k plus proches voisins, on détermine quelle est l'espèce majoritaire.

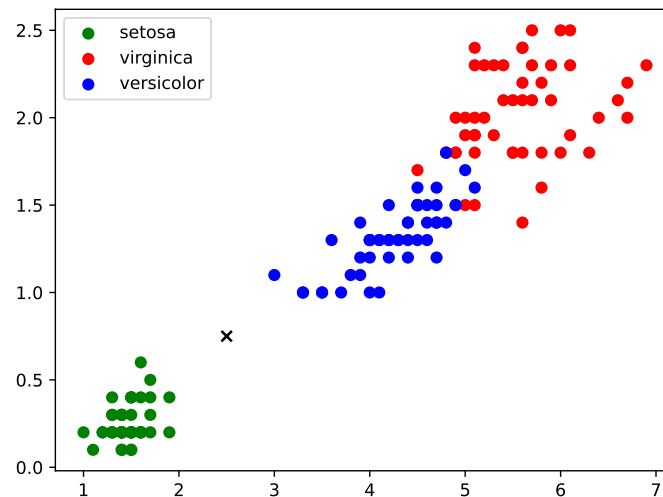


FIGURE 3 – Nuage de points des variables `petal_length` et `petal_width`

Rappels. Dans un repère orthonormé, la *distance euclidienne* entre les points $A(x_A; y_A)$ et $(x_B; y_B)$ est égale à

$$AB = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$$

Ce calcul se généralise naturellement en dimension 3 :

$$AB = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2 + (z_A - z_B)^2}$$

et en dimension quelconque :

$$AB = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$

où $(a_1, a_2, \dots, a_n)$ et $(b_1, b_2, \dots, b_n)$ sont les coordonnées de a et b dans $\mathbb{R}^n$.

Astuce. la commande `a**0.5` retourne la racine carrée de a .

1. Écrire une fonction `distance(p1, p2)` qui calcule la distance euclidienne entre les points `p1` et `p2` de $\mathbb{R}^n$.
2. Pour la suite on prendra $k = 5$ et un iris dont les dimensions sont :
 - `petal_length = 2.5`
 - `petal_width = 0.75`
 - `sepal_length = 5.8`
 - `sepal_width = 3`
 - (a) Ajouter une colonne nommée `'dist'` qui contient la distance entre l'iris dont on cherche à prédire la variété et chacune des iris du jeu de données.
 - (b) Trier les données dans l'ordre croissant suivant cette nouvelle colonne et afficher les 5 premières lignes.
Quelle est la prédiction pour l'iris considérée ?
3. Écrire une fonction `prediction_iris(new_point, data, k)` qui prend comme paramètre en entrée :
 - `new_point` : dimension de l'iris dont on souhaite prédire la variété ;
 - `data` : le jeu de données initial ;
 - `k` : le nombre de voisins les plus proche de la variété à prédire.
 Elle affiche les k plus proche voisins et retourne le nom de la variété prédite.

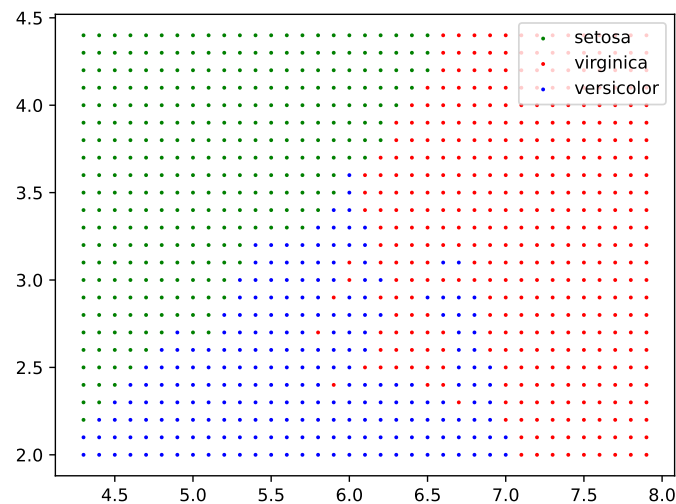


FIGURE 4 – Carte de prédiction

4. Testez votre fonction avec $k = 5$ et
 - (a) `new_point = [5.4, 2.7, 3.75, 1.2]`
 - (b) `new_point = [6.3, 3.2, 5.75, 1.75]`
5. Testez d'autres valeurs de k . Commenter.

Fonctions `pandas` utiles.

La bibliothèque `pandas` offre des facilités pour la manipulation des tables de données (`data_frame`) :

- on peut ajouter une nouvelle colonne et cette nouvelle colonne peut être exprimée en fonction des deux autres. Par exemple, ajoutons une colonne qui est la somme de la longueur des pétales et de la largeur des pétales :
`iris['somme'] = iris['petal_length'] + iris['petal_width']`
- `dataframe.sort_values(by = ['C'])` retourne un `dataframe` avec les lignes triées de telle sorte que la colonne `'C'` soit dans l'ordre croissant.
- `Series.value_counts()` retourne le nombre d'occurrence de chacune des valeurs de la series de nombres `Series` ;
- `Series.idxmax()` retourne l'index de l'élément maximum.

2.3 Carte de prédiction

Avec ces prédictions, nous pouvons créer une *carte de prédiction*.

Par exemple, si l'on regarde le nuage de points des variables `sepal_length` et `sepal_width` (voir figure 2), on peut, en faisant varier x entre le minimum `sepal_length` et son maximum et y entre le minimum et le maximum de `sepal_width`, colorer le point de coordonnées $(x; y)$ en fonction de la prédiction obtenue.

Dans ce cas, pour simplifier, on peut n'utiliser que les variables `sepal_length` et `sepal_width` pour les calculs.

On obtient la carte de la figure 4 page 5.

Sauriez-vous la reproduire ?

3 Jeu d'apprentissage et jeu de test

Pour tester notre méthode, nous pouvons séparer notre jeu de données en deux :

- une partie consacrée à l'apprentissage : ce sont les points dont nous connaissons la variété et qui sont candidats à être les plus proches voisins ;
- une partie pour le test : points dont nous voulons trouver la variété.

Nous pouvons ainsi connaître le nombre d'erreurs de prédiction sur un échantillon.

Commençons par mélanger le jeu de données et le couper en deux, par exemple, 100 iris pour la partie apprentissage et 50 pour le jeu de test.

[15]:

```
iris = pd.read_csv("iris.csv") # on recharge le jeu de données initial
T = iris.sample(n = 150).reset_index() # on mélange le jeu de données et on ré-indexe les lignes

iris_learn = T.loc[:100][:] # pour l'apprentissage
iris_test = T.loc[100:][:] # pour les tests
```

Les lignes de `iris_learn` sont indexées de 0 à 99 et celle de `iris_test` de 100 à 149.

1. En adaptant le code des parties précédentes, écrire un script qui :
 - associe à chacune des iris de `iris_test` la variété la plus probable selon la méthode des k plus proche voisins ;
 - compte le nombre d'erreurs commises.
2. Tester avec plusieurs valeurs de k .

Exemple d'utilisation. On peut, par exemple, créer la nouvelle colonne `iris_test["predict"]` et comparer avec l'a véritable variété de l'iris.

[19]:

```
ans = iris_test.query('species == predict')

print(ans.shape)
```

[19]:

(47, 7)

Trois erreurs de prédictions dans ce cas !